

Generative Project: DECODING THE VISUAL WORLD — IMAGE-TO-TEXT GENERATION USING VISION TRANSFORMERS AND LANGUAGE MODELS

Course: IE 7615 Deep Learning for AI

Focus Area: Generative Deep Learning Models

1. Overview

Generative modeling has revolutionized how AI systems create visual content from textual descriptions. This project challenges you to design and implement an **Image-to-text generative pipeline** by integrating **Transformer-based text encoders** and **CLIP based image encoders**.

Through this project, you will gain hands-on experience with *conditional generative models*, *latent representations*, and *cross-modal learning*—the key foundations of modern AI systems such as BLIP, Flamingo, and GIT (Generative Image-to-Text Transformer)

2. Objectives

By the end of this project, students will:

- Understand the architectural principles behind **Vision Transformers** and **Transformer-based language decoders**.
 - Implement or fine-tune a **conditional language model** using **image embeddings** as conditioning input.
 - Explore how image–text alignment and multimodal representations work.
 - Evaluate generated captions quantitatively and qualitatively.
 - Reflect on ethical and societal implications of generative AI.
-

3. Project Description

Core Tasks

1. Dataset Preparation

- Use a paired image–caption dataset (e.g., **COCO Captions**, **Flickr30k**, or a smaller curated set*).
- Preprocess image using **CLIP ViT-based image encoder** to extract image embedding as conditional input.
- Tokenize and preprocess captions using a **Transformer-based tokenizer** (e.g., GPT-2 tokenizer).
- Normalize and resize images for consistent input dimensions. (e.g. 244x244)

2. Model Implementation

- Use a **pre-trained language model** (e.g., GPT-2) conditioned on CLIP image embedding to generate captions.
- Experiment with:
 - **Embedding injection methods** — prefix conditioning vs. cross-attention conditioning.
 - **Decoding strategies** — greedy search, beam search, and nucleus (top-p) sampling.
 - **Temperature scaling** to control output diversity.
 - **Fine-tuning strategies** — full fine-tuning vs. prefix tuning vs. LoRA adapters.

3. Evaluation

- Quantitatively assess caption quality using:
 - **BLEU** (Bilingual Evaluation Understudy)
 - **METEOR** (Metric for Evaluation of Translation with Explicit ORdering)
 - **CIDEr** (Consensus-based Image Description Evaluation)
 - **ROUGE-L** (Recall-Oriented Understudy for Gisting Evaluation)
- Qualitatively evaluate **semantic alignment** between generated captions and source images.
- Visualize and present generated caption examples alongside their source images.

4. Analysis and Reflection

- Discuss challenges in image-text conditioning and cross-modal alignment.
 - Highlight trade-offs between caption fidelity and diversity.
 - Reflect on potential ethical concerns (e.g., dataset bias, misrepresentation of visual content, accessibility implications).
-

4. Deliverables

1. Project Report (6–8 pages)

- Introduction, background, and related work
- Methodology and model architecture
- Experimental setup and results
- Analysis and discussion
- Ethical considerations and conclusion

2. Generated Results Gallery

- Showcase examples of **input images and corresponding generated captions**.

3. Source Code Repository

- Well-documented code with clear dependencies and instructions.

4. Presentation (~10–15 minutes)

- Summarize goals, approach, and key findings.
-

5. Suggested Tools & Frameworks

- **Programming:** Python, PyTorch, Hugging Face Transformers
 - **Pre-trained Models:** CLIP(Vit image encoder), GPT
 - **Fine-tuning Methods:** LoRA (via PEFT library), Prefix Tuning
 - **Evaluation Libraries:** pycocoevalcap (BLEU, METEOR, CIDEr, ROUGE), nltk
 - **Datasets:** COCO Captions, Flickr30k, or a small subset of LAION-400M
-

6. Evaluation Criteria

Category	Description	Weight
Technical Implementation	Correctness, efficiency, and originality of model integration	30%
Experimental Results	Quality and diversity of generated captions	25%
Analysis & Discussion	Depth of analysis and reflection on findings	20%
Report Quality	Clarity, organization, and presentation	15%
Reproducibility	Code organization and documentation	10%

7. Suggested Milestones

Milestone	Timeline	Key Goals & Tasks	Deliverables
1 Milestone 1 – Project Setup & Dataset Preparation	Week 1–2	- Choose dataset (COCO Captions, Flickr30k, or curated subset) - Clean & preprocess 1k–5k image–caption pairs - Set up environment (PyTorch + Hugging Face Transformers + CLIP) - Extract CLIP image embeddings; build baseline notebook for image→embedding pipeline - Tokenize captions using GPT-2 tokenizer	1-page proposal+ front page(dataset + model + roles) - Sample 5 image→caption test runs - GitHub repo initialized
2 Milestone 2 – Model Integration & Baseline Results	Week 3–4	Integrate CLIP image encoder with GPT-2 language decoder	- Training log + early samples (5–10 images) 1-page summary of observations +front page

		• Implement embedding injection (prefix conditioning or cross-attention) • Run baseline caption generation on sample images • Experiment with decoding strategies (greedy, beam search, nucleus sampling) • Record first set of generated captions	
3 Milestone 3 – Evaluation & Analysis	Week 5	• Quantitative metrics: BLEU, METEOR, CIDEr, ROUGE-L • Qualitative alignment: input image vs. generated caption • Parameter sensitivity (temperature scaling, beam width, embedding injection method) • Compare fine-tuning strategies (full fine-tuning vs. LoRA vs. prefix tuning) • Visualize & compare results across decoding strategies	• Evaluation notebook • Tables + plots of results • Draft report (methods & results sections)
4 Milestone 4 – Final Report & Presentation	Week 7	• Final report (15–17 pages) + ethical reflection • Curate generated captions gallery (images + captions side by side) • Prepare slides & demo (15–20 min) • Submit complete code & results	• Final PDF report • Presentation slides / demo • GitHub repo with README

8. Optional Extensions

- Experiment with alternative pre-trained captioning models (e.g., **BLIP-2**, **GIT**, or **Flamingo**) and compare their outputs against your GPT-2 baseline.
- Try **different vision encoders** (e.g., ViT-L/14 vs. ViT-B/32 from CLIP) and analyze how encoder capacity affects caption quality.
- Explore **multi-sentence or paragraph-level caption generation** instead of single-sentence captions.
- Add **visual question answering (VQA)** as an extended task — condition the model on both an image and a question to generate a text answer.
- Fine-tune using **LoRA or prefix tuning** and compare against full fine-tuning in terms of performance and efficiency.
- Experiment with **domain-specific datasets** (e.g., medical imaging, satellite imagery) to test generalization of the captioning pipeline.